

PROYECTO E-COMMERCE - Construyendo paso a paso

OBJETIVOS DE APRENDIZAJE

Al final de este proyecto sabrás:

- Crear componentes reutilizables
 - Pasar datos entre componentes con props
 - Manejar listas con map()
 - Usar renderizado condicional
 - Gestionar estado complejo con useState
-

FASE 1: TU PRIMERA TARJETA DE PRODUCTO (20 minutos)

 **Objetivo:** Crear un componente que muestre UN producto

TU MISIÓN:

1. **Crea** el archivo `Components/TarjetaProducto.jsx`
2. **Analiza** esta información de producto:

javascript

```
const producto = {  
  id: 1,  
  nombre: "iPhone 15 Pro",  
  precio: 1199,  
  imagen: "https://picsum.photos/300/200",  
  disponible: true,  
  marca: "Apple"  
}
```

PREGUNTAS PARA PENSAR:

- ¿Qué props necesita recibir tu componente?
- ¿Cómo mostrarías el precio de forma destacada?
- ¿Qué pasa si el producto NO está disponible?

ESTRUCTURA MÍNIMA:

jsx

```
function TarjetaProducto({ /* ¿qué props necesitas? */ }) {  
  return (  
    <div className="tarjeta-producto">  
      /* TODO: Mostrar imagen */  
      /* TODO: Mostrar nombre */  
      /* TODO: Mostrar marca */  
      /* TODO: Mostrar precio */  
      /* TODO: Botón "Añadir" o "Agotado" según disponibilidad */  
    </div>  
  )  
}
```

RETO CSS:

Haz que tu tarjeta se vea profesional:

- Bordes redondeados
- Sombra sutil
- Hover effect
- Imagen que ocupe todo el ancho

CRITERIO DE ÉXITO:

Cuando termines, deberías poder usarla así:

jsx

```
<TarjetaProducto  
  nombre="iPhone 15"  
  precio={999}  
  disponible={true}  
  marca="Apple"  
>
```

FASE 2: LISTA DE PRODUCTOS (25 minutos)

 **Objetivo:** Mostrar VARIOS productos usando tu componente

 **DATOS PARA TRABAJAR:**

javascript

```
const productos = [  
  { id: 1, nombre: "iPhone 15", precio: 999, marca: "Apple", disponible: true },  
  { id: 2, nombre: "Samsung S24", precio: 899, marca: "Samsung", disponible: true },  
  { id: 3, nombre: "Google Pixel", precio: 699, marca: "Google", disponible: false }  
]
```

? PREGUNTAS CLAVE:

- ¿Cómo convertir un array en una lista de componentes?
- ¿Qué es la prop `key` y por qué la necesitas?
- ¿Qué pasa si el array está vacío?

TU COMPONENTE ListaProductos:

jsx

```
function ListaProductos({ productos, titulo }) {  
  return (  
    <div className="lista-productos">  
      <h2>{titulo}</h2>  
      <p>Mostrando { /* ¿cuántos productos? */ } productos</p>  
  
      { /* TODO: ¿Qué pasa si no hay productos? */ }  
      { productos.length === 0 ? (  
        /* ¿Qué mostrar? */  
      ) : (  
        <div className="grid-productos">  
          { /* TODO: Usar map() para mostrar cada producto */ }  
          { /* Recuerda la prop key! */ }  
        </div>  
      ) }  
    </div>  
  )  
}
```

RETOS MENTALES:

1. **Renderizado condicional:** ¿Cómo mostrar "No hay productos" cuando el array esté vacío?
2. **Map y key:** ¿Por qué React necesita la prop `key`? ¿Qué usar como key?
3. **Props dinámicas:** ¿Cómo pasar el título desde el componente padre?

✅ **TEST:** Tu ListaProductos debería funcionar así:

jsx

```
<ListaProductos  
  productos={misProductos}  
  titulo="Smartphones en oferta"  
>
```

FASE 3: CARRITO INTELIGENTE (30 minutos)

 **Objetivo:** Crear un sistema de carrito que guarde productos

PIENSA PRIMERO:

- ¿Dónde guardas los productos del carrito? (pista: useState)
- ¿Cómo añades un producto SIN duplicarlo?
- ¿Cómo calculas el total de productos y precios?

ALGORITMO PARA AÑADIR:

SI el producto ya está en el carrito:

- Aumentar su cantidad en 1

SINO:

- Añadirlo con cantidad = 1

PREGUNTAS CLAVE:

1. **Estado del carrito:** ¿Array de productos o array de {producto, cantidad}?
2. **Función añadir:** ¿Cómo buscar si un producto ya existe?
3. **Mostrar total:** ¿Cómo sumar todas las cantidades?

MINI-RETOS:

- **Reto 1:** Función para añadir productos
- **Reto 2:** Función para cambiar cantidad (+/-)
- **Reto 3:** Función para eliminar producto
- **Reto 4:** Calcular precio total

PISTAS:

- Usa `find()` para buscar productos existentes
- Usa `map()` para actualizar cantidades
- Usa `filter()` para eliminar
- Usa `reduce()` para sumar totales

FASE 4: FILTROS INTELIGENTES (25 minutos)

 **Objetivo:** Permitir buscar y filtrar productos

TIPOS DE FILTROS A IMPLEMENTAR:

1. **Búsqueda por texto** (nombre o marca)
2. **Filtro por categoría** (smartphones, laptops, etc.)
3. **Solo disponibles** (checkbox)
4. **Ordenar por** (precio, nombre, valoración)

PREGUNTA CLAVE:

¿Cómo combinar TODOS los filtros a la vez?

ALGORITMO DE FILTRADO:

```
productos
.filter(producto => {
  // ¿Cumple la búsqueda?
  // ¿Cumple la categoría?
  // ¿Cumple disponibilidad?
})
.sort((a, b) => {
  // ¿Cómo ordenar según la opción elegida?
})
```

MINI-EJERCICIOS:

1. **Crear estados** para cada filtro
2. **Función de búsqueda** que ignore mayúsculas/minúsculas
3. **Dropdown de categorías** dinámico
4. **Función de ordenación** por diferentes criterios

TRUCOS:

- Usa `toLowerCase()` para búsquedas
- Usa `includes()` para texto parcial
- Combina filtros con `&&` (AND lógico)

FASE 5: INTEGRACIÓN FINAL (20 minutos)

 **Objetivo:** Conectar todo en tu App principal

CONEXIONES NECESARIAS:

- Header ↔ Búsqueda
- TarjetaProducto ↔ Carrito
- Filtros ↔ Lista de productos
- Carrito ↔ Contador del Header

PREGUNTA ARQUITECTURAL:

¿Dónde vive cada estado? ¿En qué componente?

REGLA DE ORO:

El estado debe vivir en el componente padre común más cercano

TU CHECKLIST FINAL:

- ☐ Buscar productos desde el Header
 - ☐ Añadir productos al carrito desde las tarjetas
 - ☐ Ver contador actualizado en el Header
 - ☐ Filtrar y ordenar productos
 - ☐ Abrir/cerrar carrito modal
 - ☐ Cambiar cantidades en el carrito
 - ☐ Calcular totales correctamente
-

CRITERIOS DE ÉXITO

★ BÁSICO (Aprobado):

- Mostrar lista de productos
- Añadir al carrito
- Ver total de productos

★ ★ INTERMEDIO (Notable):

- Búsqueda funcional
- Filtros por categoría
- Carrito con cantidades

★ ★ ★ AVANZADO (Sobresaliente):

- Todos los filtros funcionando
- Carrito completo con modal
- Diseño responsivo y profesional

SI TE ATASCAS...

HERRAMIENTAS DE DEBUG:

1. **console.log()** es tu amigo
2. **React DevTools** para ver estados
3. **Divide y vencerás**: haz una cosa cada vez

PREGUNTAS DE AUTOAYUDA:

- ¿Qué error aparece en la consola?
- ¿El estado se está actualizando?
- ¿Las props llegan al componente hijo?
- ¿Estoy usando las llaves `{}` correctamente?

PIDE AYUDA CUANDO:

- Llevas más de 10 minutos atascado
- No entiendes un error específico
- Tu código "debería funcionar" pero no funciona

CONCEPTOS QUE PRACTICAS

- ✓ **useState** con diferentes tipos de datos
- ✓ **Props** y comunicación entre componentes
- ✓ **map()** para renderizar listas
- ✓ **filter()** y **find()** para trabajar con arrays
- ✓ **Renderizado condicional** (&&, ternario)
- ✓ **Event handling** (onClick, onChange)
- ✓ **Componentes reutilizables**
- ✓ **Arquitectura de aplicación React**

¡Al final tendrás un e-commerce funcional construido desde cero! 